LAM: Learning Adaptation Model – Product Requirements Document

Oct 18, 2026 · @Mr.Forever

LAM is an independent, versioned service that turns structured student-learning evidence into a learner state and a recommended next teaching action (v0.1, draft).

1. Overview and goals

LAM is not a chatbot. Qrie and other learning apps consume it through a stable API, and it stays separate from Qrie's frontend, classroom logic and LLM provider.

Problem: tutors often cannot tell what a student understands, why they struggle, or whether an intervention worked.

Goals

- Estimate per-concept mastery, separating assisted from unassisted performance.
- Diagnose likely causes of errors, or ask a diagnostic question when evidence is weak.
- Recommend the next teaching action and schedule retrieval reviews.
- Track which strategies work, with uncertainty.
- Serve any LMS through a standard API and SDK.

Primary metric: unassisted mastery gain per learning minute.

Supporting metrics: delayed retention, transfer, and recommendation acceptance rate.

Non-goals: generating lessons or chat replies, authentication, billing, school administration, fixed learner-type labels, and training on raw conversations.

Consumers: the Qrie classroom and tutor (recommendations and state), third-party LMS apps (REST and SDK), learners and teachers (dashboards via the host app), and the LAM team (audit logs, evaluation, rollout).

2. Scope by release

The immediate target is M0 and M1. Neural-network training waits until measured evidence shows it beats a simpler baseline.

Release	Focus	Contents
M0	Service foundation	Repo scaffold, typed schemas, auth, PostgreSQL migrations, versioned API, event ledger, tests, SDK skeleton
M1	Deterministic learner model	Per-concept state, BKT mastery, assisted/unassisted tracking, misconception evidence, rule-based next action
M2	Closed loop with Qrie	Diagnosis, adaptive teaching, independent assessments, delayed retrieval, persistent state
M3	Personalization	Strategy effectiveness, uncertainty calibration, policy evaluation
M4	Shared intelligence	Approved multi-student data, offline evaluation, model versioning, controlled rollout and rollback

3. Architecture

A request flows through six stages: API gateway and auth, evidence validation, diagnosis engine, learner state engine, adaptive decision engine, and recommendation API.

Private data layer: student profiles, learning event ledger, curriculum/concept graph (consumed through a stable interface), and strategy effectiveness.

Shared intelligence (separate and governed): approved dataset, then training and evaluation, then a versioned shared model. Training is never an automatic side effect of API requests, and the shared model never stores identifiable histories.

Responsibility split

- Shared model: general patterns across students.
- Student profile: one learner's mastery, misconceptions, retention and strategy outcomes.
- Decision engine: combines both with curriculum constraints and the current context.
- LLM: external and replaceable, invoked by the consuming tutor when the decision engine requests it.

4. Input and output contracts

LAM accepts structured evidence, not raw transcripts. A missing signal means "unknown", never "negative".

Input group	Fields	Status
Identity and context	`event_id`, `student_id`, `concept_id`, `item_id`, `timestamp`	Required (MVP)
Response	`correctness`, `attempt_number`	Required (MVP)
Question	`difficulty`, `concept_ids`	Recommended
Assistance	`hints_used`, `assistance_level`	Recommended
Timing and confidence	`response_time_ms`, `self_reported_confidence`	Recommended
Teaching context	`strategy_id`, `intervention_id`	Recommended
Curriculum	`subject`, `grade`, prerequisite graph reference	Recommended
Historical state	Learner-state version or server-side profile reference	Recommended

Identity must come from trusted credentials, not a client-supplied `student_id`. Working steps, voice, handwriting and engagement signals arrive in later releases, each with its own privacy controls.

Output

- `learner_state`: mastery estimate, unassisted mastery, assisted performance, uncertainty
- `diagnosis`: candidate causes with confidence, plus `needs_confirmation`
- `next_action`: type, strategy, assistance level, target concept
- `review`: required flag and recommended interval
- A short rationale (no chain-of-thought) and `state_version`

Scores need explicit definitions and must not imply scientific precision until calibrated and validated.

Example input (illustrative)

```
{
  "event_id": "evt_1042",
  "student_id": "student_123",
  "concept_id": "fractions.unlike_denominators",
  "item_id": "q_892",
  "timestamp": "2026-10-18T14:30:00Z",
  "correctness": false,
  "attempt_number": 2,
  "hints_used": 1,
  "assistance_level": 1,
  "response_time_ms": 42000,
  "self_reported_confidence": 0.8,
  "strategy_id": "visual_explanation_v1"
}
```

Example output (illustrative)

```
{
  "student_id": "student_123",
  "concept_id": "fractions.unlike_denominators",
  "learner_state": {
    "mastery_estimate": 0.55,
    "unassisted_mastery": 0.42,
    "assisted_performance": 0.78,
    "uncertainty": 0.30
  },
  "diagnosis": {
    "candidates": [
      {"type": "conceptual_misunderstanding", "label": "Denominator
addition", "confidence": 0.65}
    ],
    "needs_confirmation": true
  },
  "next_action": {
    "type": "ask_diagnostic_question",
    "strategy": "socratic_question",
    "assistance_level": 2,
    "target_concept": "fractions.unlike_denominators"
  },
  "review": {"required": true, "recommended_interval_hours": 48},
  "state_version": 12
}
```

5. Functional requirements

ID	Requirement	Description
FR-01	Learner state	Versioned, auditable per-concept mastery, uncertainty, misconceptions, assisted/unassisted performance, retention
FR-02	Diagnosis	Candidate causes with confidence; ask a diagnostic question when evidence is insufficient
FR-03	Adaptive decisions	Choose among independent practice, metacognitive prompts, hints, analogies, worked examples, prerequisite remediation, direct explanation
FR-04	Mastery verification	Confirm mastery only with independent assessment evidence
FR-05	Sequencing	Use mastery, prerequisite gaps, misconception severity, forgetting risk, curriculum importance, difficulty
FR-06	Retrieval scheduling	Track last practiced, history, memory strength and next review; update on success or failure
FR-07	Strategy effectiveness	Outcomes per concept and learner, accounting for sample size, uncertainty and confounders
FR-08	Model lifecycle	Versioned training, offline evaluation, rollout, monitoring, rollback
FR-09	Explainability	Action, rationale, evidence and uncertainty; no internal chain-of-thought
FR-10	Integration	Versioned APIs, SDKs, idempotent event submission, state retrieval, schema migration policy

6. Non-functional requirements

The service must be independently deployable, testable and replaceable without changing Qrie's core classroom architecture.

- **Isolation:** server-side authorization and tenant-aware access on every endpoint. A student ID in the URL is not authorization.

- **Privacy:** data minimization, defined consent, retention, deletion and training eligibility.
- **Reliability:** idempotent ingestion, concurrency-safe state updates, recoverable failures.
- **Performance:** latency targets set from real classroom needs; training stays off the request path.
- **Versioning:** API schemas, learner-state schema, decision policies and shared models versioned independently.
- **Observability:** request IDs, model versions, decision reasons, errors and latency, without logging sensitive student content.
- **Portability:** HTTP/JSON, a documented Python SDK and a TypeScript client.
- **Safety:** validate recommendations; escalate on high uncertainty or repeated struggle.

7. API surface and tech stack

The v1 API is small and versioned. Every endpoint validates the caller's permission to access the learner.

Endpoint	Purpose
`POST /v1/events`	Ingest a learning event and update state
`GET /v1/students/{id}/state`	Retrieve the authorized learner profile
`POST /v1/students/{id}/diagnose`	Evaluate a suspected misconception or knowledge gap
`POST /v1/students/{id}/recommendations`	Select the next best learning action
`POST /v1/students/{id}/assessments`	Record assessment evidence
`GET /v1/students/{id}/reviews`	Retrieve due or upcoming retrieval tasks
`GET /v1/health`	Check service health
`GET /v1/models/current`	Report the active shared model version

Proposed SDK interface (not yet existing):

```
lam.record_event(event)
lam.get_learner_state(student_id)
lam.recommend_next_action(student_id, concept_id)
lam.record_assessment(result)
lam.get_due_reviews(student_id)
```

Layer	Choice	Reason
API	FastAPI + Pydantic	Typed contracts, easy integration
Decision engine	Python rules and deterministic policies	Easy to test, explain and debug
Mastery model	Bayesian Knowledge Tracing (or another validated approach)	Models knowledge updates from interaction evidence
Concept graph	Consume Qrie's graph via a stable interface	Avoid duplicating curriculum infrastructure
Storage	PostgreSQL	Transactional state updates and event records
Background jobs	Task queue only when volume requires it	Keep heavy processing off the request path
ML	scikit-learn first, PyTorch only when justified	Avoid premature complexity
Model tracking	Versioned artifacts and evaluation reports	Reproducible deployment and rollback
Integration	REST + Python SDK + TypeScript client	Works with Qrie and other apps
Deployment	Docker + CI	Reproducible, independent releases

BKT is a reasonable starting point for concept-level mastery but is not a complete diagnosis or teaching-strategy model. Effectiveness must be evaluated, not assumed.

8. Boundaries, acceptance criteria and risks

What LAM must not do

- Generate every lesson or reply itself.
- Treat every wrong answer as a misconception.
- Assign permanent learner types (visual, auditory, etc.).
- Count assisted success as independent mastery.
- Train on all raw conversations, or update global weights after every answer.
- Infer fatigue, boredom or emotion with unjustified confidence.
- Own authentication, billing or school administration.

Acceptance criteria (v1)

1. A retried event updates the correct concept state exactly once.
2. A likely misconception triggers a diagnostic question or intervention.
3. Assisted and unassisted mastery are stored separately.
4. A later independent assessment can change mastery and the review schedule.
5. Recommendations respect prerequisites.
6. Student A cannot read or modify Student B's profile.
7. Qrie can use the API without importing LAM's internal packages.
8. Models can be evaluated and rolled back without losing learner history.
9. Decisions and state updates are reproducible in known test scenarios.
10. Evaluation shows improvement on a predefined outcome against a baseline before any effectiveness claim.

Risks: BKT misfit on some concepts, confounded strategy-effectiveness data, privacy and consent for shared training, and over-trusting uncalibrated scores.

Open questions

- Latency targets
- Single-tenant vs multi-tenant
- Concept-graph interface format
- Consent model for M4 training data
- Baseline for the effectiveness evaluation

Ownership boundary: LAM owns the learner model, evidence processing, diagnosis, adaptive decisions, review scheduling and the shared-model lifecycle. Qrie owns the student UI, content delivery, lesson rendering, tutor conversations, voice and school workflows. They communicate only through versioned contracts.